

Note per il Corso di Programmazione

Antonio Caruso

Dipartimento di Matematica e Fisica '*Ennio De Giorgi*'
Palazzo Fiorini

`mailto://antonio.caruso@unisalento.it`

`http://bilbo.unisalento.it/antonio`

Settembre 2015

Indice

Prefazione	v
1 Rappresentazione dei Numeri	1
1.1 Numeri Interi	1
1.1.1 Conversione tra basi	6
1.1.2 Aritmetica in Binario, Ottale, Esadecimale	8
1.1.3 Numeri Interi Negativi	10
1.2 Numeri Razionali e Reali	15
1.2.1 Aritmetica in Virgola Fissa	17
1.2.2 Numeri Floating-Point - IEEE 754	18
1.3 Testi, Immagini, Musica	20

Prefazione

Queste Note fanno parte del materiale didattico per il corso di Programmazione presso l'Università degli Studi del Salento (Laurea Triennale in Matematica). Il corso prevede 6 crediti ETCS, ogni settimana vi sono di norma 2 ore di lezione e 2 ore di lezione/esercitazione presso il laboratorio di calcolo del Dipartimento. Gli argomenti inclusi sono:

- **Rappresentazione dell'Informazione** all'interno del Calcolatore, con un' enfasi particolare per la parte numerica.
- **Architettura del Calcolatore:** Logica digitale, Circuiti Combinatori/Sequenziali, Modello di Calcolo di Von Neumann. Architettura e Funzionamento del sistema centrale CPU-RAM.
- **Algoritmi e Modelli di Calcolo:** Concetti, Modelli e Formalizzazioni della nozione di Algoritmo/Programma.

Non è richiesta conoscenza preliminare sugli argomenti trattati nel corso. E' necessaria una conoscenza di base di Matematica Discreta e di Algebra.

Si ringraziano in anticipo coloro che, qualora trovassero delle inesattezze o errori nelle note, vogliano cortesemente segnalarcele via email, o direttamente a lezione.

License

This work is licensed under the *Creative Commons Attribution-ShareAlike 3.0 Unported License*. To view a copy of this license, visit

<http://creativecommons.org/licenses/by-sa/3.0/>

Capitolo 1

Rappresentazione dei Numeri

I do hate sums!! There is no greater mistake than to call arithmetic an exact science. There are ... hidden laws of Number which it requires a mind like mine to perceive. For instance, if you add a sum from the bottom up, and then again from the top down, the result is always different!

- M. P. LA TOUCHE (1878)

Vedete, non esiste nulla (amati studenti in Matematica) che sia così fastidioso nella pratica matematica, e più molesto che dover calcolare moltiplicazioni, divisioni, e radici quadrate o cubiche di numeri molto grandi. A parte la noia e lo spreco di tempo, queste sono spesso soggette a errori scivolosi. Ho cominciato a considerare nella mia mente, con quale arte io possa rimuovere tali ostacoli.

- JOHN NEPAIR [NAPIER] (1916)

1.1 Numeri Interi

Il primo approccio con la Matematica è *contare*. Il contare porta alla definizione dei numeri interi, attualmente scritti attraverso l'uso di una *notazione posizionale decimale* che come vedremo è stata introdotta in Europa intorno al A.D. 1202¹ da parte di Leonardo Pisano (detto il Fibonacci). Gli Interi e le operazioni elementari tra essi, come somma, sottrazione, moltiplicazione e divisione sono studiati in quella parte della matematica chiamata *Aritmetica*; Nonostante il lettore possa trovare strano dover rivedere concetti che ritiene associati, vedremo come l'Aritmetica meriti senz'altro un ulteriore approfondimento.

Queste note introdurranno una diversa prospettiva sul perché scriviamo i numeri in decimale. Vedremo come la scelta di una rappresentazione numerica è strettamente legata al modo con cui svolgiamo abitualmente le operazioni aritmetiche, cioè gli *algoritmi* usati per il calcolo di somma, sottrazione, etc. Questi aspetti sono di interesse sia per comprendere all'interno del corso di Programmazione come viene rappresentata l'informazione dentro il computer, sia per chi dovrà svolgere attività didattica nelle scuole elementari; evidenziando la natura essenzialmente *algoritmica* dell'Aritmetica.

Siamo interessati in particolare allo studio della *rappresentazione* dei numeri. In questa prospettiva dobbiamo chiaramente sottolineare la differenza tra due entità: un numero, inteso astrattamente come *valore* e la sua *rappresentazione* (o *numerales*) intesa come sequenza di segni che servono appunto a scrivere o comunicare un certo valore. Cominciamo quindi, con il distinguere un valore numerico, ad esempio l'intero 1234, da una delle sue tante possibili *rappresentazioni*, termine che vuol indicare *il modo in cui scriviamo* quel valore. Noi scriviamo (e pensiamo) i numeri allo stesso modo, usando un sistema

rappresentazione
vs.
valore

¹Quindi per migliaia di anni il modo con cui si pensava ad i numeri, li si scriveva e soprattutto si faceva Aritmetica è stato molto diverso da come noi siamo abituati a fare sin da piccoli.

posizionale *decimale* pertanto il modo più comune di scrivere 1234 è ovviamente: '1234'², ma lo stesso numero sarebbe stato scritto come 'MCCXXXIV' se vivessimo nell'antica Roma. Una rappresentazione numerica è una serie di convenzioni più o meno formalizzabili che descrivono come si *scrivono* i numeri, cioè una serie di regole '*sintattiche*' che possono variare nel tempo e nello spazio.

La rappresentazione posizionale, ci consente di scrivere qualunque numero (naturale) come una *stringa*³ di *cifre* (*digit* in inglese). Vale il seguente Teorema:

Teorema 1 (Rappresentazione dei numeri Naturali in base B).

Sia $B > 1$ un intero che chiameremo *base* (o *radice*) della rappresentazione, e $\mathcal{C} = \{c_0, c_1, \dots, c_{B-1}\}$ un insieme di simboli distinti dette *cifre della rappresentazione* (in inglese *digit*). Ogni cifra è associata ad un valore intero tra 0 e $B - 1$.

Allora $\forall n \in \mathbb{N}$, esiste ed è unica la sequenza di cifre: $(c_{k-1}c_{k-2} \dots c_1c_0)_B$ ed il suo valore è:

$$n = c_{k-1}B^{k-1} + c_{k-2}B^{k-2} + \dots + c_1B + c_0 = \sum_{i=0}^{k-1} c_i B^i \quad (1.1)$$

Il sistema *decimale* ha $B = 10$ e $\mathcal{C} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ ed il lettore lo usa quotidianamente fin dalle elementari. Con $B = 2$ come vedremo il sistema è detto binario e lo tratteremo approfonditamente, con $B = 3$ abbiamo numeri in ternario, con $B = 4$ in quaternario, con $B = 8$ in ottale. Possiamo avere anche $B > 10$, ad esempio con $B = 16$ abbiamo il sistema esadecimale, le cifre comunemente usate nel sistema decimale non bastano a rappresentare i simboli delle cifre esadecimali, e si usa normalmente $\mathcal{C} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ con la convenzione che $A = 10, B = 11, \dots, F = 15$.

Esempio 1: $(1234)_{10} = 1 \cdot 10^3 + 2 \cdot 10^2 + 3 \cdot 10 + 4 = 1234$. $(1234)_8 = 1 \cdot 8^3 + 2 \cdot 8^2 + 3 \cdot 8^1 + 4 \cdot 8^0 = (668)_{10}$. $(11010010)_2 = 1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^4 + 1 \cdot 2^1 = (210)_{10}$. $(1ABF)_{16} = 1 \cdot 16^3 + 10 \cdot 16^2 + 11 \cdot 16 + 15 = 6847_{10}$. $(4D2)_{16} = 4 \cdot 16^2 + 13 \cdot 16 + 2 = 1234_{10}$. $(1234)_3$? La sequenza non rispetta le condizioni del Teorema, non ha quindi molto senso valutarla poiché se $B = 3$ si ha $\mathcal{C} = \{0, 1, 2\}$ ed i simboli 3,4 non hanno alcun significato in tale sistema.

Nel seguito se la base B è chiara dal contesto non useremo le parentesi e la base come pedice, pertanto $(1001001)_2$ sarà scritto semplicemente come 1001001 se è chiaro che stiamo trattando numeri in base 2. Noi usiamo la base 10 per cui scriviamo 1298 e non $(1298)_{10}$, ed il valore ad esso associato è l'intero 1298. Quest'ultima osservazione che può sembrare banale è invece opportuna in quanto è chiaro che data una sequenza di cifre, il valore ad essa associato dipende dal sistema di numerazione in cui ci troviamo, cioè dalla base usata, la stessa sequenza infatti avrà valori diversi per basi diverse.

Condizione per
l'esistenza e unicità
della rappresentazione

La condizione $0 \leq c_i \leq B - 1$ è essenziale per garantire l'esistenza e l'unicità della rappresentazione. Nell'Esempio precedente abbiamo scritto che non ha senso valutare $(1234)_3$, proviamo ad essere più precisi: consideriamo $B = 2$ e supponiamo vi siano 4 cifre: $\mathcal{C} = \{0, 1, 2, 3\}$ vediamo subito che l'unicità della rappresentazione non vale più poiché $13 = 3 \cdot 2^2 + 1 \cdot 2^0$ ma anche $2 \cdot 2^2 + 2 \cdot 2^1 + 1 \cdot 2^0$. Quindi se usiamo *troppe* cifre, perdiamo l'unicità, allo stesso modo se usiamo *poche* cifre perdiamo l'esistenza di una rappresentazione, ad esempio se in base 10 usassimo solo le cifre $0, \dots, 8$ avremmo infiniti numeri naturali non rappresentabili (9, 19, ...).

Il valore k in (1.1) è il numero di cifre, le singole cifre prese da $\mathcal{C}$ hanno significato diverso in base alla loro posizione, ecco perché il sistema è detto posizionale. Il peso è

²le due scritture sono diverse per enfatizzare che 1234 è il valore, ed '1234' solo un modo particolare per scrivere tale valore.

³sequenza di caratteri giustapposti

proporzionale ad una potenza della base via via crescente da destra a sinistra. La prima cifra $c_{k-1} \neq 0$ viene detta *più significativa* poiché associata alla potenza della base più alta, corrispondentemente la cifra c_0 o cifra delle *unità* viene chiamata anche *meno significativa*. Vale la pena ricordare che nella didattica della Matematica nelle classi primarie (elementari) è usuale dare dei numeri come 1214 e chiedere di scomporli nelle loro cifre e nei pesi della base attraverso la dizione seguente:

1214 = un migliaio, due centinaia, una decina, e quattro unità.

in questo modo i bambini apprendono rapidamente che le cifre rappresentano valori differenti a secondo della loro posizione, e che tale valore è proporzionale ad un opportuna potenza della base (a questo livello identificate con opportune espressioni: migliaia, centinaia, etc). Dimostriamo adesso il Teorema:

Dimostrazione: Fissiamo la base B e dimostriamo esistenza e unicità di (1.1) per induzione su n .

Chiaramente 1 è sempre rappresentabile in uno ed un solo modo⁴. Adesso supponiamo induttivamente che ogni intero $1, \dots, n-1$ sia rappresentabile in modo univoco.

Consideriamo l'intero n , chiamiamo $d = n \bmod B$, allora $d < B$ è una delle cifre disponibili. Per induzione $n' = (n-d)/B$ è unicamente rappresentabile, diciamo ad esempio così:

$$\frac{n-d}{B} = d_0 + d_1 B + d_2 B^2 + \dots$$

Allora chiaramente

$$n = d + \frac{n-d}{B} B = d + d_0 B + d_1 B^2 + d_2 B^3 + \dots$$

è una rappresentazione di n che usa solo le cifre disponibili, e l'esistenza è dimostrata. Adesso supponiamo che n abbia due rappresentazioni diverse avremmo:

$$\begin{aligned} n &= a_0 + a_1 B + a_2 B^2 + \dots \\ &= c_0 + c_1 B + c_2 B^2 + \dots \end{aligned}$$

Ma poichè da sopra, c_0 e a_0 sono uguali a $d = n \bmod B$, allora sono uguali, quindi la differenza deve essere nelle altre cifre. Ma allora il numero n' ha due rappresentazioni diverse, ma questo contraddice l'ipotesi induttiva. $\square$

Svolgendo le operazioni in (1.1) in base 10 abbiamo un modo semplice per convertire un naturale dalla base $B \neq 10$ alla base $B = 10$. Abbiamo già visto, nell'Esempio 1, come convertire numeri binari, ottali o esadecimali in decimale. La stessa formula ci mostra come i numeri siano strettamente collegati al concetto di polinomio: la somma in (1.1) può essere vista come un polinomio di grado $k-1$, i cui coefficienti sono le cifre dell'intero, valutato nel punto $x = B$.

In seguito tratteremo in modo approfondito l'uso della base due. Se consideriamo $B = 2$ e $\mathcal{C} = \{0, 1\}$ il sistema è chiamato *binario* proprio perché le cifre a disposizione sono solo due. Le singole cifre vengono anche chiamate *bit* dalla contrazione di *Binary digIT* (che non è altro che l'inglese di "Cifra Binaria").

E' nozione comune che le informazioni all'interno di un calcolatore digitale sono rappresentate in binario, ovviamente come *lunghe* sequenze di bit. Questo rende tale sistema importante per la comprensione di come viene codificata l'informazione dentro un calcolatore. L'uso della base due è motivato dal fatto che è molto semplice misurare una piccola variazione di potenziale in un circuito, e definire così due *stati* distinti: possiamo ad esempio associare a 0V (Volt) il valore 0 ed 1 al livello +3V. Se un circuito può

Sistema Binario

Bit

Sistema Binario,
Circuiti Bistabili,
Hardware del
Calcolatore

⁴perché?

assumere in modo stabile due livelli (non necessariamente quelli indicati sopra), è detto *bistabile* e consente di rappresentare una cifra binaria. La procedura con cui convertiamo l'informazione in sequenze di bit viene detta comunemente *digitalizzazione*. In informatica il termine bit, indicherà quindi una cifra (0/1) di un numero binario, ma in alcuni contesti si indicherà con lo stesso termine il circuito bistabile usato per memorizzare un bit.

Quando progettiamo un circuito stabiliamo una volta per tutte la dimensione dei fili in ingresso ed in uscita da esso. Pertanto, siamo particolarmente interessati a studiare le proprietà dei sistemi posizionali con un numero *finito* di cifre.

Consideriamo quindi il valore di k come *costante*; il numero di cifre degli interi che possiamo scrivere con questo vincolo è al massimo k e l'intervallo dei valori rappresentabili risulterà chiaramente limitato. Il minimo è ovviamente 0 (se $\forall i c_i = 0$). Il massimo si ha quando ogni cifra assume il valore massimo, cioè se $\forall i c_i = B - 1$ quindi:

Interi in base B con
massimo k cifre:
Range = $[0, B^k - 1]$

Osservazione 1.

Con k cifre in base B dalla (1.1) si ha:

$$\sum_{i=0}^{k-1} (B-1)B^i = (B-1) \sum_{i=0}^{k-1} B^i = (B-1) \frac{1-B^k}{1-B} = B^k - 1 \quad (1.2)$$

Ed i numeri rappresentabili sono gli interi nell'intervallo $[0, B^k - 1]$.

dove la penultima equivalenza deriva dalla somma finita (con indici generalizzati) di una serie geometrica, i.e. per $a, b \in \mathbb{N}$, si ha:

$$\sum_{i=a}^b x^i = \frac{x^a - x^{b+1}}{1-x}$$

Isomorfismo tra
numeri di k cifre e
 k -tuple

Un altro modo per ottenere (1.2) senza molti conti è il seguente: invece di scrivere le cifre tramite una sequenza di caratteri (una stringa) possiamo scriverle ognuna come componente di una k -tupla⁵, quindi la rappresentazione in (1.1) diventa $(c_{k-1}, c_{k-2}, \dots, c_0)$ ⁶ allora ogni numero con k cifre può essere visto come un elemento di $\mathcal{C}^k$ (il prodotto cartesiano di $\mathcal{C}$ per se stesso, k volte), ma la cardinalità di quest'ultimo insieme è banalmente $|\mathcal{C}^k| = |\mathcal{C}|^k = B^k$, quindi vi sono B^k configurazioni diverse ed ognuna rappresenterà un intero diverso; Poichè iniziamo a contare da 0 il valore massimo rappresentabile sarà $B^k - 1$.

Esempio 2: In base 10 con 4 cifre abbiamo i valori da 0 a 9999, cioè l'insieme $\mathcal{C}^4 = \{(c_3, c_2, c_1, c_0) \mid 0 \leq c_i \leq 9\} = \{(0, 0, 0, 0), \dots, (9, 9, 9, 9)\}$. Vi sono quindi $|\mathcal{C}^4| = 10^4 = 10000$ numeri possibili, da 0 a $9999 = 10^4 - 1$.

Se usiamo la base $B = 2$ i valori possibili vanno da 0 a $1111 = 2^3 + 2^2 + 2^1 + 2^0 = 2^4 - 1 = 15$, con 8 bit abbiamo $2^8 = 256$ valori, tra 0 e $(11111111)_2 = 255$. In esadecimale con 4 cifre abbiamo $16^4 = 65536$ valori possibili, da 0 a $\text{FFFF} = 65535$.

Il numero di cifre per
 n è $k = \lceil \log_B n \rceil$

Quindi se fissiamo il numero di cifre, l'intervallo di rappresentazione aumenta all'aumentare della base. Sia $n \in \mathbb{N}$, per rappresentare n dobbiamo avere k tale che $B^k - 1 \geq n$, quindi $k = \log_B n + 1 \approx \lceil \log_B n \rceil$.

⁵Una k -tupla è una generalizzazione del concetto di coppia, tripla, quadrupla... con k componenti. In generale è un elemento del prodotto cartesiano di k insiemi.

⁶La presenza delle virgole non è una differenza da poco: in (1.1) usiamo una stringa di cifre, i.e. le cifre sono *giustapposte* senza separatori, in una tupla le componenti sono chiaramente separate dalle virgole. Le due rappresentazioni sono matematicamente diverse anche se ovviamente isomorfe.

Esempio 3 (Numero di cifre di un intero in base B): L'ultima relazione ci dice che la lunghezza in cifre di un numero intero n in decimale è $\lceil \log_{10} n \rceil$, il che è un'ovvia conseguenza della (1.1). Prendiamo $n = 123456789$, esso ha $\lceil \log_{10} n \rceil = \lceil 8,0915 \rceil = 9$ cifre, poiché $\log_2 n = 8,0915 / \log_{10} 2 = 26,87$, in binario serviranno 27 bit. Infatti come vedremo successivamente: $123456789 = (111010110111100110100010101)_2$.

Lo Zero, l'Unario, i Numeri Romani

Perché usiamo la notazione posizionale, ed in particolare il sistema decimale? E come sono stati rappresentati i numeri interi prima della diffusione del sistema decimale?

Sappiamo che in passato alcune popolazioni hanno usato in ambiti specifici numeri in base 60 o in base 12, ma è evidente che la preferenza per il sistema decimale sia attribuibile alla presenza di dieci dita, che ha reso molto naturale il suo apprendimento. Resta da capire perché preferire un sistema posizionale rispetto ad esempio a quello romano?

Cominciamo a rispondere alla domanda precedente, rivedendo come sono stati scritti o meglio 'segnati' i numeri fin dagli albori del tempo. Il modo primitivo per rappresentare delle quantità intere è stato l'uso di pietre (*calculi*) che rappresentavano le unità, dallo stesso termine deriva l'etimologia di *calcolare*.

Dal punto di vista astratto possiamo vedere questo modo di rappresentare i numeri come quello di un uomo che chiuso in una cella segna i giorni con delle tacche su un muro. Questo sistema usa un solo simbolo 'I' per rappresentare le unità, può rappresentare un qualunque numero intero semplicemente *giustapponendo* un pari numero di simboli, quindi $2 = II$, $3 = III$, $4 = IIII$, $5 = IIIII$, etc. Questo sistema viene chiamato *unario*. Per quanto esso sia estremamente semplice, anche per svolgere le operazioni aritmetiche⁷, esso è affetto da un problema fondamentale: la lunghezza dei numeri cresce *esattamente* come il numero da rappresentare: ci vogliono n 'I' per rappresentare n . Questo rende impraticabile la rappresentazione di numeri anche relativamente piccoli, (il lettore provi a immaginare di scrivere in unario 53). Nei sistemi posizionali la rappresentazione è esponenzialmente più corta, in quanto il numero di cifre come abbiamo visto è solo $\log n$ (Per cinquantatré bastano due cifre), e questa differenza è forte anche rispetto al più semplice sistema posizionale, quello binario, che tra tutti è quello che richiede più cifre.

Sistema Unario

Assodata la difficoltà di utilizzo di un sistema unario, l'unico modo per migliorare le cose è quello di aumentare il numero di simboli. Ma a questo punto dobbiamo stabilire che valore dare a diversi simboli, e in quali circostanze.

I Romani hanno sviluppato un sistema a *sostituzione* in cui ogni volta che la rappresentazione di un valore diventa troppo lunga, si associa tale valore ad un diverso simbolo: poiché $5 = IIIII$ è già troppo lungo, introduciamo $V = 5$, ed essendo quattro, uno prima di cinque, abbiamo $4 = IV$, naturalmente 6 è uno dopo 5 quindi VI e con lo stesso identico sistema possiamo introdurre $X = 10$, $L = 50$, $C = 100$, $D = 500$, $M = 1000$ e rappresentare $2015 = MMXV$ con solo quattro cifre (notare l'assenza dello zero). Il sistema consente di scrivere numeri senza l'esplosione esponenziale della loro lunghezza come nell'unario, ma è comunque affetto da numerosi problemi.

Numerazione Romana

Il numero di simboli non può essere costante come nei sistemi posizionali, ma deve crescere, cosa accadrebbe se vogliamo rappresentare 10^9 in romano? servono altri simboli dopo M. Più importante ancora è la difficoltà dell'aritmetica: Come possiamo descrivere le operazioni di somma o sottrazione? In effetti tali operazioni non sono per nulla facili da descrivere, e sicuramente sono decisamente più complesse rispetto all'aritmetica decimale. I romani in pratica usavano fare le somme usando un *abaco* (vedi Figura 1.1 accanto) solo per numeri molto piccoli.



Figura 1.1: Abaco

Gli storici della matematica credono che l'uso della notazione posizionale decimale e l'introduzione dello *zero* siano da attribuire a *Brahmagupta* un matematico indiano nel 620AD. Questo sistema raggiunge l'Europa passando prima attraverso la Spagna (allora Musulmana) e diffondendosi in Italia. L'italiano Leonardo di Pisa, chiamato anche il Fibonacci, scopre questo

⁷Il lettore provi a definire in modo chiaro, come svolgere le operazioni aritmetiche in unario.

Muhammad ibn Musa
al-Khwarizmi

sistema di numerazione e l'aritmetica ad esso associata lavorando nella bottega del padre, nella città di Bugia (una città Africana nella regione corrispondente all'attuale Algeria). E' attraverso gli scritti di Muhammad ibn Musa al-Khwarizmi, Abu Kamil e altri matematici arabi che Fibonacci apprende il nuovo sistema, e lo diffonde in Europa dove viene presto adottato insieme all'uso dello zero.

Zero in
Matematica

E' evidente l'importanza sia pratica (all'interno del sistema di numerazione) sia concettuale dello zero. Il fatto che dovesse esistere un *numero* che indicasse un *assenza* di quantità, è un concetto che richiese secoli prima di essere assimilato. Lo zero è fondamentale all'interno dei sistemi posizionali, lo studente oramai capisce la motivazione di fondo: in un intero come 5005, gli zeri sono essenziali per *spostare il peso di una cifra senza aggiungerne complessivamente al valore* (vedi i due 5). Una volta introdotto è necessario capire come le operazioni aritmetiche si comportano su esso, e naturalmente si nota subito il problema legato alla divisione. Il fatto che l'operazione di divisione *non fosse definita* (oggi diremmo che essa costituisce una funzione parziale sugli interi, vedi Appendice ??) è qualcosa che contrariava molti matematici. Non parliamo delle resistenze dei filosofi a dare un nome a una quantità per rappresentare....un assenza di quantità.

1.1.1 Conversione tra basi

Di seguito si vedrà come convertire da una rappresentazione in base b ad una in base B . Poiché noi siamo abituati a svolgere operazioni in base 10, per facilitare il cambio di base, trasformeremo prima un valore dalla base b di partenza alla base 10, e poi convertiremo il valore decimale ottenuto in base B . Come visto anche nell'esempio 1, la formula in (1.1) ci consente di convertire un numero da una base qualunque alla base 10. Pertanto resta da capire come convertire da base 10 ad un'altra base generica B . Osserviamo che dato un intero n , l'essenza della dimostrazione del Teorema 1 è la riscrittura dell'equazione (1.1) come⁸:

$$n = \sum_{i=0}^{k-1} c_i B^i = (((c_{k-1}B + c_{k-2})B \dots + c_2)B + c_1)B + c_0$$

i.e. $1234 = ((1 \cdot 10 + 2) \cdot 10 + 3) \cdot 10 + 4$

Conversione in base B,
tramite divisioni
ripetute per B ed
estrazione dei resti.

quindi la cifra meno significativa c_0 può essere ottenuta dividendo (usiamo la divisione intera) n per B e prendendo il resto. Quindi abbiamo $c_0 = n \bmod B$ ed il valore della divisione $n' = n/B$ è un intero formato dalle rimanenti cifre di n . Possiamo ripetere il procedimento su n' fin quando il valore della divisione è diverso da zero estraendo così tutte le cifre.

Esempio 4: Consideriamo $n = 12345$, abbiamo $n' = n/10 = 1234$ e $n \bmod 10 = 5$. Ripetendo l'operazione di modulo su n' otteniamo la cifra 4, e così via. Se invece di 10 usiamo il valore della base B , otterremo valori tra 0 e $B - 1$, che non sono altro che le cifre di n espresso in base B .

Convertiamo ad esempio $n = 12345$ in binario, scrivendo per ogni divisione il risultato della stessa + il resto.

12345/2 = 6172 + 1	96/2 = 48 + 0
6172/2 = 3086 + 0	48/2 = 24 + 0
3086/2 = 1543 + 0	24/2 = 12 + 0
1543/2 = 771 + 1	12/2 = 6 + 0

⁸Questo è un altro esempio del parallelismo tra numeri e polinomi, infatti questo modo di scrivere i polinomi è detto formula di Horner ed è molto usata per velocizzare la valutazione del polinomio in un punto.

$$\begin{array}{ll}
771/2 = 385 + 1 & 6/2 = 3 + 0 \\
385/2 = 192 + 1 & 3/2 = 1 + 1 \\
192/2 = 96 + 0 & 1/2 = 0 + 1 \quad \uparrow
\end{array}$$

e poiché l'ultima cifra è la più significativa si ha che 12345 in binario è 11000000111001. Proviamo a convertire 58 in esadecimale, osserviamo che $58/16 = 3$ e $58 \bmod 16 = 10$ quindi $(58)_{10} = (3A)_{16}$. Convertiamo 12345 in ottale:

$$\begin{array}{l}
12345/8 = 1543+1 \\
1543/8 = 192+7 \\
192/8 = 24+0 \\
24/8 = 3+0
\end{array}$$

Quindi $12345 = (30071)_8$.

Il lettore è invitato adesso a confrontare la sequenza dei valori nei calcoli sopra con i valori ottenuti nella conversione in binario. Si nota facilmente che poiché $8 = 2^3$, ogni passo nella conversione in ottale, costituisce tre passi nella conversione decimale, è facile verificare che questo dipende direttamente dalle proprietà della rappresentazione in base. In ottale in effetti, ogni cifra è un valore tra 0 e 7, se vogliamo rappresentare questi valori in binario serviranno $\log 8$ bit, cioè 3 bit per ogni cifra ottale.

Questo ci suggerisce un modo veloce per passare da una base b ad una base $B = b^i$. Se si passa da b a B si tratta di *raccogliere* ogni i cifre e rappresentarle come un'unica cifra in base B . Se viceversa si passa dalla base B alla base b allora ogni cifra si deve *espandere* in i cifre della base b .

Conversione da
binario ad ottale ad
esadecimale

Esempio 5: Nell'esempio precedente $n = 12345$ e in binario vale 11000000111001, per passare velocemente in ottale, basta osservare che

$$\underbrace{011}_3 \underbrace{000}_0 \underbrace{000}_0 \underbrace{111}_7 \underbrace{001}_1$$

Notiamo che si inizia a raggruppare dalle cifre *meno* significative, cioè da destra a sinistra, nel caso in cui come nell'esempio si hanno meno cifre di quelle da raggruppare, si devono aggiungere degli zeri, che posti a destra delle altre cifre (in figura in grassetto) non ne cambiamo il valore. Otteniamo quindi esattamente il valore calcolato precedentemente. Lo stesso vale per la conversione in esadecimale, invece di usare le divisioni, poiché $16 = 2^4$ possiamo fare così:

$$\underbrace{0011}_3 \underbrace{0000}_0 \underbrace{0011}_3 \underbrace{1001}_9$$

quindi $12345 = (3039)_{16}$, verifichiamolo: $3 \cdot 16^3 + 3 \cdot 16 + 1 = 12345$. Possiamo ovviamente fare lo stesso nella direzione contraria, cioè trasformare $(3039)_{16}$ in binario, espandendo le cifre esadecimali, nel corrispondente valore binario, basta *girare* il senso delle parentesi graffe nello schema sopra, e cancellare i due zeri frontali (ridondanti).

In Informatica l'uso della notazione ottale e soprattutto esadecimale consente di rappresentare lunghe sequenze di bit in modo decisamente più compatto, un numero binario di 32 bit infatti richiederà solo $8 = 32/4$ cifre esadecimali.

Il metodo esposto sopra produce le cifre nell'ordine opposto rispetto al modo in cui noi scriviamo, cioè da destra a sinistra, dalla meno alla più significativa. E' possibile ottenere la sequenza a partire da c_{k-1} fino a c_0 , cioè nell'ordine in cui noi scriveremo il numero? Dalla (1.1), abbiamo che per estrarre la cifra c_{k-1} dobbiamo dividere il valore dell'intero

che stiamo considerando, chiamiamolo n , per B^{k-1} . Ad esempio se vogliamo estrarre 3 da 3240 dobbiamo dividerlo per $1000 = 10^3$, quindi dobbiamo sapere quante sono le cifre di n , ma questo si ottiene calcolando $k = \lceil \log_{10} n \rceil$, notiamo che le restanti cifre sono contenute in $3240 \bmod 10^3 = 240$.

Conversione tramite divisioni per potenze decrescenti della base

L'algoritmo pertanto è il seguente: Se si vuole convertire il numero decimale n in base B si calcola $k = \log_B n$, si calcola $c_{k-1} = n/B^{k-1}$ e $n' = n \bmod B^{k-1}$, si diminuisce k e si applica lo stesso metodo a n' .

Esempio 6: Convertiamo 3240 in esadecimale usando il metodo sopra:

$$\begin{aligned} \lceil \log_{16} 3240 \rceil &= 3 \\ 3240/16^2 &= 12 = c_2, & 3240 \bmod 16^2 &= 168 \\ 168/16^1 &= 10 = c_1, & 168 \bmod 16 &= 8 \\ 8/16^0 &= 8 = c_0. \end{aligned}$$

Quindi 3240 in esadecimale è CA8.

1.1.2 Aritmetica in Binario, Ottale, Esadecimale

L'aritmetica in base $B \neq 10$ segue esattamente le stesse regole di quella con i numeri decimali.

Cominciamo con l'operazione più semplice, il calcolo del successore di un intero n , valutare cioè $n + 1$, questo ci consentirà di imparare a contare in una generica base B . Il lettore svolge questa operazione in modo *meccanico* poiché l'ha appresa in tal modo fin dalle elementari.

Contare in Base B

Ma come possiamo formalizzare esattamente tale operazione? E' facile verificare che il seguente Algoritmo è la risposta giusta: esaminate le cifre del numero n dalla meno significativa alla più significativa fino a trovare la prima cifra minore di $B - 1$ se esiste, cambiate mentre si scorre verso sinistra ogni $B - 1$ trovato in 0. Se si trova una cifra minore di $B - 1$ si incrementa, altrimenti aggiungete una cifra pari ad 1 a destra del numero.

Lo studente provi a le seguenti operazioni in base 10, seguendo l'algoritmo espresso sopra: $139 + 1 = 140$, $99 + 1 = 100$. Notate inoltre che l'algoritmo presentato non è altro che l'algoritmo della somma imparato alle elementari ma presentato in modo leggermente diverso, senza usare i riporti per far scivolare il valore del +1 verso sinistra. Lo stesso algoritmo si può quindi usare per contare in una base.

Contare in basi diverse

Esempio 7 (Conteggio e Somma in base B): Contiamo in binario: $0, 1, (1 + 1) = 10, (10 + 1) = 11, (11 + 1) = 100, 100 + 1 = 101, 101 + 1 = 110$, etc.
Contiamo in base 3: $0, 1, 2, (2 + 1) = 10, 11, 12, 20, 21, 22, 100$, etc.
Contiamo in base 4: $0, 1, 2, 3, (3 + 1) = 10, 11, 12, 13, 20, 21, 22, 23, 30$, etc.
Contiamo in base 8: $0, \dots, 7, (7 + 1) = 10, 11, 12, 13, 14, 15, 16, 17, 20, \dots$, etc.

Sommare in Binario

Le operazioni aritmetiche quindi si svolgono in modo identico a quello imparato alle elementari. Sommiamo $31 + 12$ in binario. Si ha: $31/2 = 15 + 1, 15/2 = 7 + 1, 7/2 = 3 + 1, 3/2 = 1 + 1$, quindi $31 = (1111)_2$ con $12 = (8 + 4) = 1100_2$

$$\begin{array}{r} 1 \quad \leftarrow \text{resti} \\ 1111+ \\ 1100 \\ \hline 11011 \end{array}$$

che deriva dal fatto che: $1 + 0 = 1$, e $1 + 1 = 10$ quindi si riporta un resto a destra, sopra ad esempio abbiamo segnato il resto riportato nella somma tra le cifre in seconda

colonna. Nella prima colonna si ha quindi $1 + 1 + 1 = 11$ e quindi il risultato presenta una cifra in più. Se il numero di cifre è limitato, ad esempio se nell'esempio sopra il numero di bit fosse limitato a 4, il risultato sarebbe stato trocato a 1011 ed il bit scartato viene usato proprio per segnalare questa situazione chiamata *errore di overflow* (trabocco).

Overflow

La sottrazione tra due numeri a, b con $a > b$ avviene come in decimale, nel caso in cui il sottraendo è maggiore del minuendo, avviene un prestito, tenendo conto del peso delle cifre: se prendiamo in prestito da sinistra un unità, diventano due unità (in generale B unità) prestate a destra.

Sottrazione in binario

Esempio 8 (Sottrazione Binaria): Calcoliamo $25 - 14$, abbiamo $25 = 11001$ e $14 = 1110$ quindi

4	3	2	1	0	indice colonna
	2	<u>2</u>	2		resti
<u>1</u>	<u>1</u>	0	0	1	—
0	1	1	1	0	
0	1	0	1	1	

In effetti $(1011)_2 = 2^3 + 2 + 1 = 11 = 25 - 14$.

Vediamo in dettaglio: nella colonna 0 a destra abbiamo $1 - 0 = 1$, nella colonna 1 abbiamo $0 - 1$ e non possiamo farlo senza prestiti. Quindi chiediamo in prestito dalla colonna 2 che però è a zero, quindi proseguiamo e chiediamo un prestito dalla colonna 3, dove abbiamo un 1 (abbiamo evidenziato con una sottolineatura i valori da cui prendiamo in prestito). Il prestito della colonna 3 diventa un due nella colonna 2 (guardate la riga con i resti), da questo prediamo in prestito un unità, che diventa un 2 nella colonna 1. A questo punto possiamo completare l'operazione in colonna 1, con $2 - 0 - 1 = 1$, nella colonna 2, abbiamo $0 - 1$ ma dobbiamo considerare un unità di resto rimasta e quindi otteniamo $1 - 1 = 0$. Nella colonna tre, abbiamo $0 - 1$ perché abbiamo preso in prestito, quindi nuovamente prendiamo in prestito dalla colonna 4 ottenendo $2 - 1 = 1$, a questo punto la colonna quattro è $0 - 0 = 0$. Il procedimento è complicato, soprattutto per la gestione dei resti, anche se in fondo si tratta dello stesso procedimento che usiamo con numeri decimali.

Sottrazione
Regole sui Prestiti

Se $a < b$, l'operazione (sui Naturali) di sottrazione non è definita e si verifica una situazione di *errore di underflow*.

Underflow

La moltiplicazione segue le stesse regole usate per moltiplicare numeri decimali, si vuol sottolineare qui solo un aspetto, di natura prettamente *computazionale*. Sappiamo che la moltiplicazione di $a \cdot b$ è semplicemente la somma ripetuta di a con se stesso per b volte (e per la proprietà di commutatività, anche la somma ripetuta di b con se stesso a volte). Come mai non usiamo questo procedimento per svolgere la moltiplicazione? (provate adesso a svolgere 145×34 senza calcolatrice). Invece, come appreso alle elementari, moltiplichiamo a per le singole cifre di b ($145 \times 3 = 435$, $145 \times 4 = 580$) e scriviamo questi prodotti parziali incolonnandoli in modo relativo alla cifra corrispondente usata in b , poi sommiamo questi prodotti parziali ottenendo un valore ($435 \times 10 + 580 = 4930$). Come mai questo valore corrisponde al prodotto?

Moltiplicazione
Numero di Operazioni

Lasciamo questo quesito al lettore (che probabilmente non si era mai posto tale problema) qui invece sottolineiamo che il motivo per cui svolgiamo la moltiplicazione in questo modo è perché è *efficiente*. Comporta cioè molte meno operazioni del metodo basato sulle somme ripetute, dove per *operazioni* intendiamo le somme e prodotti tra singole cifre: lo studente provi a *contarle* nei due casi.

1.1.3 Numeri Interi Negativi

Nella Sezione precedente, lo studente dovrebbe aver compreso come rappresentare un numero naturale in una base arbitraria, come convertire tra basi diverse, ed in particolare in binario, e nelle basi ad esso correlate (ottale, esadecimale). Inoltre, dovrebbe saper contare in una qualunque base e svolgere le operazioni di somma, sottrazione, moltiplicazione e divisione. Infine, dovrebbe conoscere le relazioni che legano il valore di un numero rispetto al numero di cifre che lo compongono nelle diverse basi.

In questa Sezione ci occuperemo di capire come rappresentare i numeri interi negativi, e quindi passare da $\mathbb{N}$ a $\mathbb{Z}$. Nell'uso comune i numeri negativi sono rappresentati aggiungendo un simbolo '-' davanti alla loro rappresentazione come numeri naturali positivi. Quindi abbiamo 1234, e -1234, e questa è sicuramente la rappresentazione più semplice. Dal punto di vista astratto tale rappresentazione è perfettamente compatibile con la Definizione in (1.1), infatti la applichiamo al sistema decimale senza problemi.

L'obiettivo di questa sezione è quindi leggermente più complesso: Non ci serve semplicemente una notazione per i numeri negativi, in base arbitraria, ma una notazione che li rappresenta senza usare simboli aggiuntivi rispetto all'insieme $\mathcal{C}$ definito in (1.1). In altri termini: Poiché i computer memorizzano tutto in forma binaria, il sistema di rappresentazione che useremo per i numeri negativi deve essere compatibile con questa scelta, deve cioè consentire di *codificare* interi positivi o negativi in modo che alla fine la rappresentazione del numero sia sempre e solo costituita da una stringa di bit. I sistemi di rappresentazione che vedremo, per quanto definiti per basi arbitrarie, hanno una reale applicazione solo in binario.

Modulo e Segno

Il primo modo, molto semplice per codificare i numeri negativi è notare che i simboli '+' e '-' per il segno, sono solo due, quindi possono essere rappresentati usando un bit: associamo ad esempio 0 = '+' e 1 = '-', e mettiamo questo bit davanti alla rappresentazione del valore assoluto del numero.

Definizione 1 (Modulo e Segno).

La rappresentazione *con Modulo e Segno* di un intero n in binario con k cifre è la seguente:

$$n = (c_{k-1}c_{k-2}\dots c_1c_0)_2 = (-1)^{c_{k-1}} \cdot \sum_{i=0}^{k-2} c_i 2^i$$

Notiamo quindi che il segno è rappresentato nella cifra più significativa che non viene a far parte del *valore* dell'intero come le altre cifre. Come si vede dagli indici della sommatoria il range di rappresentazione è dato dagli interi nell'intervallo $[-2^{k-1} - 1, 2^{k-1} - 1]$.

Esempio 9: Il valore -12 sarà ottenuto prima calcolando il valore del modulo (valore assoluto) $12 = (1100)_2$ e poi aggiungendo un bit di segno messo ad 1, quindi $(-12 = 11100)$. Notiamo che abbiamo usato 5 bit, infatti con soli 4 bit, dovendo usarne una per il segno, avremmo a disposizione solo 3 bit per il valore assoluto, e quindi il massimo valore rappresentabile sarebbe $111 = 7$. Con 5 bit possiamo quindi rappresentare i valori interi in $[-2^4 - 1, 2^4 - 1] = [-15, 15]$. In generale si deve sempre prestare attenzione a fissare il numero di bit in modo che l'intervallo di rappresentazione sia sufficientemente grande da poter rappresentare gli interi di nostro interesse.

Nella Tabella 1.1 vediamo un confronto tra l'interpretazione di numeri binari con 4 bit nei due sistemi visti finora. Notiamo che nella prima colonna abbiamo aggiunto degli zeri

Numeri Interi in
Modulo e Segno, con
 k bit: Range =
 $[-2^{k-1} - 1, 2^{k-1} - 1]$

Binario	Posizionale	Modulo e Segno	Scostamento (a 7)	Complemento a 2
0000	0	+0	-7	+0
0001	1	+1	-6	+1
0010	2	+2	-5	+2
0011	3	+3	-4	+3
0100	4	+4	-3	+4
0101	5	+5	-2	+5
0110	6	+6	-1	+6
0111	7	+7	+0	+7
1000	8	-0	+1	-8
1001	9	-1	+2	-7
1010	10	-2	+3	-6
1011	11	-3	+4	-5
1100	12	-4	+5	-4
1101	13	-5	+6	-3
1110	14	-6	+7	-2
1111	15	-7	+8	-1

Tabella 1.1: Numeri binari di 4 bit: valore rispetto alla notazione posizionale (naturali), e confronto dei valori nei vari sistemi per numeri negativi.

davanti ad ogni numero in modo da evidenziare sempre il numero di cifre che abbiamo fissato ($0000 = 0$). Come si vede chiaramente ogni riga della tabella, nelle colonne dalla seconda in poi, presenta un modo diverso di *interpretare* una certa sequenza di bit (della prima colonna). Quindi chiedersi quale sia il valore di una certa sequenza di bit senza sapere quale sia il sistema di rappresentazione usato è insensato.

Ad esempio nella prima colonna il valore della sequenza di bit 1010 rappresenta il valore 10 se si interpreta come un intero senza segno, se invece la sequenza di bit rappresenta un valore scritto nella notazione con modulo e segno allora essa vale -2 . E' responsabilità di chi ha scritto la sequenza di bit, sapere come interpretarla.

La rappresentazione in modulo e segno è semplice, ma presenta vari problemi che hanno portato alla sua scomparsa nell'uso pratico. Notiamo dalla Tabella che esiste sia il valore $+0$ che il valore -0 , e questo comporta uno spreco poiché due diverse sequenze di bit, sono di fatto associate allo stesso valore. Inoltre la presenza di un bit di segno separato, rende più complicate le operazioni aritmetiche di somma e sottrazione (ma facilita le moltiplicazioni/divisioni). Vediamo un esempio di sottrazione:

Esempio 10: Proviamo a calcolare $25 - 14$: nel sistema con modulo e segno, per rappresentare 25 serviranno almeno 6 bit per la rappresentazione. Infatti $2^{6-1} - 1 = 31 > 25$, mentre $2^{5-1} - 1 = 15$ ed il valore 25 non sarebbe rappresentabile con 5 bit.

Abbiamo quindi $25 = 011001$ e $-14 = 101110$. Adesso nasce un problema: la sottrazione ha regole complicate sui prestiti, ma in algebra $a - b = a + -b$ quindi proviamo a usare la somma tra 25 e -14 :

$$\begin{array}{r}
 11 \quad \leftarrow \text{resti} \\
 011001 + \\
 101110 \\
 \hline
 ?00111
 \end{array}$$

L'operazione di somma, non si avvicina affatto al risultato corretto: infatti $25 - 14 = 11$ su 6 bit, con segno e modulo è rappresentato come 001011. Quello che stiamo dicendo in pratica è che il sistema di rappresentazione con modulo e segno non gode della proprietà: $a - b = (a) + (-b)$, che ci consentirebbe di trattare le sottrazioni come somme. L'operazione di sottrazione andrebbe svolta come nell'Esempio 8, con le sue complicate regole sui prestiti.

Modulo e Segno
problemi con l'ALU

L'osservazione sopra va intesa alla luce, di quello che vedremo nel Capitolo successivo sull'ALU, cioè sul circuito che internamente al Processore svolge le operazioni aritmetiche. Se adottassimo la notazione con modulo e segno, dovremmo avere una parte circuitale che confronta i segni, per controllare se sono discordi/concordi, e due circuiti diversi, per la somma e per la sottrazione da usare nei due casi. Come vedremo esistono rappresentazioni che possiedono la proprietà algebrica $a - b = (a) + (-b)$, quindi necessitano in hardware solo di un circuito per la somma. Ecco uno dei motivi principali per cui la notazione con modulo e segno non è usata all'interno del calcolatore.

Rappresentazione in Scostamento

Un altro modo di rappresentare i numeri negativi è la rappresentazione in scostamento (offset) rispetto ad un valore. Si usa una semplice traslazione per consentire di associare a numeri negativi, una rappresentazione che normalmente sarebbe associata ad un valore positivo.

Il ragionamento per definire questa rappresentazione è molto semplice. I valori rappresentabili con k cifre in base B tramite (1.1) sono gli interi in $[0, B^k - 1]$.

Definizione 2.

Sia $M > 0$ intero, la rappresentazione *in Scostamento ad M* di un intero n con k cifre è la seguente:

$$n = (c_{k-1} \cdots c_0)_B = \sum_{i=0}^{k-1} c_i B^i - M$$

Rappresentazione in
Scostamento
Intervallo di
Rappresentazione
in Binario:
 $[-2^{k-1} - 1, 2^{k-1}]$

allora il range di rappresentazione sarà: $[-M, B^k - M - 1]$. Di solito si considera $M = B^k/2 - 1$ se vogliamo circa metà di valori positivi e metà negativi. Il range dei valori rappresentati andrà dal minimo $-B^k/2 + 1$ fino al massimo $B^k - 1 - (B^k/2 - 1) = B^k/2$ per $B = 2$ si ha l'intervallo: $[-2^{k-1} - 1, 2^{k-1}]$. Ad esempio, con $k = 8$ bit, abbiamo i valori tra $[-2^7 + 1, 2^7] = [-127, 128]$. Nella Tabella 1.1 vediamo un esempio con 4 bit, e range $[-7, 8]$. Notiamo che il bit più significativo resta associato al segno ma con il significato opposto al caso precedente, se il numero è negativo il bit è a *zero*, positivo se vale uno, e lo zero sotto questa interpretazione è un *meno zero* poichè $0 = 0111$ ed il bit alto vale zero. Notiamo anche che i numeri positivi non conservano la loro rappresentazione rispetto alla notazione senza segno, e questo è una debolezza di questo sistema.

Esempio 11: I numeri decimali con al massimo 2 cifre vanno da 0 a 99. In decimale con scostamento a $M = -10^2/2 + 1 = -49$ otteniamo gli interi nell'intervallo $[-49, 50]$. Il simbolo 0 deve quindi essere interpretato come $0 - 49 = -49$, e allo stesso modo si ha: $1 = -48$, $49 = 49 - 49 = 0$ e $99 = 99 - 49 = 50$.

In Binario con otto bit e scostamento $M = 127$ abbiamo i valori interi nell'intervallo: $[-127, 128]$. Se prendiamo quindi un numero 11011011, il suo valore si ottiene convertendolo in decimale: $11011011 = 219$ e sottraendo 127 quindi si ha $(11011011) = 92$. Viceversa se avessimo dovuto *codificare* il valore di 92, avremmo prima calcolato $92 + 127$ e successivamente convertito tale valore in binario.

Questo tipo di rappresentazione è usata non tanto per rappresentare i valori interi (negativi o positivi), ma per un uso più specifico: rappresentare l'esponente nella notazione in standard IEEE-754 per i numeri in virgola mobile, come vedremo nella sezione a loro dedicata.

Esponente in
IEEE-754

Rappresentazione in Complemento alla Base

La rappresentazione *in Complemento* è l'ultima che presenteremo ed è da molti anni, quella realmente utilizzata all'interno del calcolatore per rappresentare tutti gli interi. Illustriamo subito le proprietà di cui essa gode, rispetto alle altre rappresentazioni viste finora:

1. Rispetta la proprietà algebrica: $a - b = a + -b$. Quindi le sottrazioni, possono essere svolte come semplici somme.
2. Come nel modulo e segno, il bit più alto individua il segno del numero, zero se positivo, uno se negativo. Quindi un test come $a > 0$ o $a < 0$ richiede il controllo del valore di un singolo bit.
3. I numeri positivi sono associati alla stessa sequenza di bit della notazione dell'equazione (1.1). Quindi la parte positiva della codifica corrisponde perfettamente a quanto visto per i naturali (vedi Tabella 1.1).

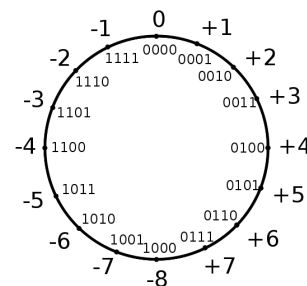
La rappresentazione in complemento alla base sfrutta una proprietà fondamentale di ogni sistema numerico *con un numero finito di cifre*. Se infatti usiamo al massimo k cifre, (abbiamo ripetuto oramai numerose volte che) il massimo valore rappresentabile è $B^k - 1$. Notiamo quindi che il valore B^k *non è rappresentabile*. In particolare cosa accade se calcoliamo $B^k - 1 + 1$ avendo solo k cifre?

Esempio 12: In decimale con tre cifre abbiamo:

$$\boxed{9} \boxed{9} \boxed{9} + 1 = 1 \boxed{0} \boxed{0} \boxed{0}$$

E poiché l'uno a sinistra non può essere scritto, abbiamo $10^3 = 1000 \equiv 0$. Lo studente di matematica attento ha già capito che in pratica la somma lavora *modulo* 1000. Ad esempio: con tre cifre decimali, il valore di -45 è 55 , perché se calcoliamo $45 + 55$ si ha $100 \equiv 0$.

Consideriamo quindi l'anello Z_{B^k} e le sue classi di resto. Ad esempio la classe $[1] = 1 + (B^k)\mathbb{Z}$ indica tutti gli interi che sono congrui ad uno modulo B^k , e così via. Tipicamente si usano come rappresentanti le classi $[0] \dots [B^k - 1]$, ma questo è solo una scelta, che può essere cambiata. In particolare, se vogliamo lavorare con valori negativi possiamo scegliere come rappresentanti i valori da $[-B^{k-1}]$ a $[B^{k-1} - 1]$.



Definizione 3 (Complemento alla Base).

Dato $n \in \mathbb{N}$, e fissato il numero di cifre della rappresentazione k , definiamo *complemento alla base* l'intero n' che soddisfa $n' = B^k - n$.

ad esempio in decimale con due cifre, il complemento a *dieci* di 45 è $55 = 100 - 45$. Nella Tabella 1.1 vediamo un esempio di rappresentazione con 4 bit, in *complemento a due*, e nella figura accanto, osserviamo l'interpretazione modulare del sistema.

Complemento a Due

Osserviamo facilmente che questo sistema soddisfa le proprietà elencate nei punti sopra: Il bit più significativo è zero se il numero è positivo, uno se negativo (punto 2); Il complemento a due di $5 = 0101$ ad esempio è $16 - 5 = 11 = 1011 = -5$, come si

vede in tabella. Vale quindi che $0101 + 1011 = 0$ (punto 1). Inoltre, i valori positivi sono rappresentati dalle stesse stringhe di bit usate per rappresentare i naturali (i numeri positivi nella quinta colonna sono uguali a quelli della seconda) quindi vale il punto 3.

Possiamo adesso definire il valore di una sequenza di cifre in complemento alla base come:

Definizione 4 (Complemento alla Base).

La rappresentazione in *Complemento alla Base* di un intero n con k cifre è la seguente:

$$n = (c_{k-1}c_{k-2} \dots c_1c_0)_2 = (-c_{k-1}B^{k-1}) + \sum_{i=0}^{k-2} c_i B^i$$

Complemento ad Uno

Mostriamo inoltre come calcolare il complemento alla base senza effettuare esplicitamente la sottrazione $B^k - n$. Poichè siamo particolarmente interessati al sistema binario, lavoriamo con $B = 2$. Il complemento a due di n è $2^k - n$ che scriviamo come $2^k - 1 - n + 1$. Definiamo *complemento ad uno* il valore $m = 2^k - 1 - n$. In generale avremo un *complemento alla base -1* definito come $m = B^k - 1 - n$ ed il complemento alla base è uguale a $m + 1$. Adesso notiamo che il complemento alla base -1 è facilmente calcolabile senza svolgere sottrazioni, poichè il valore di $B^k - 1$ è sempre costituito in ogni base da una sequenza di $k - 1$ cifre tutte uguali a $B - 1$, esse sono quindi sempre maggiori di ogni cifra in n e la sottrazione nel calcolo del complemento ad uno, può essere fatta senza considerare prestiti e analizzando solo una cifra alla volta. In binario questo corrisponde all'operazione di *complemento* o *not* dei bit, dove $\text{not } 0 = 1$, e $\text{not } 1 = 0$. Gli esempi chiariranno meglio il significato di quanto detto:

Esempio 13: Calcoliamo il complemento a due di 45 (-45).

Primo metodo: Prima di tutto per rappresentare 45 in questo sistema dobbiamo avere almeno 7 bit, infatti con 6 bit il massimo valore rappresentabile è $2^{6-1} - 1 = 31 < 45$, con 7 abbiamo $2^6 - 1 = 63 > 45$. A questo punto sappiamo che $-45 = 2^7 - 45 = 83$ e 83 in binario è 1010011 (usando uno dei metodi in 1.1.1).

Secondo Metodo: Questo metodo non userà sottrazioni, ed è quello usato davvero in hardware dai circuiti dell'ALU. Sappiamo che servono 7 bit, abbiamo $45 = 32 + 8 + 4 + 1 = 101101$, quindi 0101101. Adesso notiamo che il complemento ad uno di 45 è dato da $2^7 - 1 - 45$, e $2^7 - 1 = 127 = 1111111$. Calcoliamo $1111111 - 0101101$, notiamo che in ogni passo $1 - 1 = 0$ e $1 - 0 = 1$ quindi la sottrazione corrisponde all'operazione di *complementazione binaria*, o *negazione bit-a-bit*, in cui ogni 0 diventa 1 e ogni 1 diventa 0. Quindi il complemento ad 1 di 45 è 1010010 (notiamo che è essenziale sapere su quanti bit si sta lavorando e aggiungere sempre gli zeri di fronte, prima di complementare). A questo punto il complemento a due è $1010010 + 1 = 1010011 = 83$.

Mostriamo adesso alcune operazioni di somma con addendi di segno diverso e le loro particolarità:

Esempio 14: Calcoliamo $50 - 36$, e $36 - 50$, le operazioni sono presentate nella seguente tabella, con la riga dei resti sopra gli addendi.

$50 - 36 :$

$$\begin{array}{r} 11\ 11\ 11\ 1 \\ 00110010 + \\ 11011011 \\ \hline 100001110 \end{array}$$

 $36 - 50 :$

$$\begin{array}{r} 11\ 11 \\ 00100100 + \\ 11001101 \\ \hline 11110010 \end{array}$$

Cominciamo con l'operazione a sinistra: 50 in binario è 110010, su 6 bit senza segno. Ma lavorando in complemento a due, quel valore sarebbe negativo (bit alto ad uno), quindi *dobbiamo sempre, fissare il numero di cifre con cui lavoriamo*, ad esempio usiamo 8 bit (un byte). Quindi abbiamo $50_{10} = 00110010$ (bit alto a zero, correttamente poichè $50 > 0$).

Adesso calcoliamo il complemento a due di 36: il modulo è $32 + 4 = 00100100$, complementiamo tutti i bit, 11011011, e la somma da il risultato corretto è $00001110 = 14 > 0$, con un bit di overflow da trascurare (non segnala una condizione di errore). Notiamo che abbiamo usato il complemento ad uno come operando, ma abbiamo anche messo ad 1 il resto sulla prima cifra, trasformando quindi il complemento ad uno, nel complemento a due, come parte dello svolgimento dell'addizione. Nella parte destra, vediamo come svolgere l'operazione invertendo gli operandi, quindi $36 - 50$. Il secondo operando è il complemento ad uno di 50 come nel primo caso, ed il risultato è correttamente -14 in complemento a due. Nel caso in cui i valori sono dello stesso segno, l'operazione si svolge allo stesso modo, ma può capitare che il risultato sia fuori dal range di rappresentazione (maggiore di 127 o minore di -128), questo è segnalato dal bit di overflow che sarà di segno diverso rispetto al segno degli operandi.

Riassumendo: Lo studente adesso conosce tre modi diversi per rappresentare valori negativi: la rappresentazione in modulo e segno, in scostamento, ed in complemento alla base. Egli deve essere in grado di convertire un intero decimale in una qualunque di queste rappresentazioni specialmente nel caso binario, ed in particolare per il sistema in complemento a due. Deve anche conoscere gli intervalli di rappresentazione in funzione del numero di bit disponibili.

1.2 Numeri Razionali e Reali

La sezione precedente ha trattato i numeri Naturali e gli Interi, adesso vediamo come rappresentare numeri non interi, quindi in $\mathbb{Q}$ o $\mathbb{R}$. Notiamo che i valori di $\mathbb{Q}$ potrebbero essere rappresentati come coppie di interi p, q con $\text{mcd}(p, q) = 1$. Questa notazione presenta alcuni vantaggi: il valore della frazione è rappresentato senza perdere informazioni (senza errori di approssimazione), ma le operazioni aritmetiche sono complesse. Naturalmente questa notazione non sarebbe utilizzabile per $\mathbb{R}$. Preferiamo quindi trattare in modo uniforme gli elementi di $\mathbb{Q}$ e $\mathbb{R}$, parleremo in generale della rappresentazione di un numero reale, sapendo che se è un intero esso ricade in quanto descritto nelle sezioni precedenti altrimenti la sua rappresentazione è basata su una semplice estensione del Teorema 1:

Teorema 2.

Sia $B > 1$ la base del sistema di rappresentazione, $\mathcal{C}$ un insieme di B cifre distinte, allora $\forall x \in \mathbb{R}$, l'espansione in base B di x è una sequenza (infinita) di cifre $c_{k-1}c_{k-2}\dots c_0.c_{-1}c_{-2}\dots$ con $c_i \in \{0, \dots, B-1\}$ tali che:

$$x = \sum_{i=0}^{k-1} c_i B^i + \sum_{i=1}^{\infty} c_{-i} B^{-i} \quad (1.3)$$

Nella (1.3) abbiamo una prima sommatoria che corrisponde alla parte intera di x , e la seconda che corrisponde alla parte frazionaria. Le cifre vengono scritte a destra di un punto che separa le potenze della base positive da quelle negative: es. $234.345 = (2 \cdot 10^2 + 3 \cdot 10 + 4) + (3 \cdot 10^{-1} + 4 \cdot 10^{-2} + 5 \cdot 10^{-3})$. Le cifre della parte frazionaria possono essere infinite poiché se x è un razionale periodico o un irrazionale, la sua espansione decimale non è finita. Inoltre l'unicità è garantita solo se imponiamo delle equivalenze come $0.999999 \dots = 1^9$.

Errore di Troncamento

All'interno del calcolatore useremo una rappresentazione con un numero finito di cifre (bit), quindi dovremo in generale fissare la lunghezza della parte frazionaria e se necessario *troncare* il vero valore di x . Questo introduce un *errore di troncamento* o di *rappresentazione* inevitabile; lo studio di come tali errori influenzino le operazioni che usano questi numeri porta alla nascita del *Calcolo Numerico*.

Riguardo alla conversione tra basi diverse, abbiamo trattato la gestione della parte intera nella sezione precedente, prima di entrare nel dettaglio della rappresentazione interna al calcolatore mostriamo come convertire in particolare da decimale a binario la parte frazionaria. Usando lo stesso ragionamento fatto per la parte intera, dobbiamo *estrarre* il valore delle cifre frazionarie, da c_{-1} in poi. Notiamo che

$$B(x - \lfloor x \rfloor) = c_{-1} + \sum_{i=2}^{\infty} c_{-i} B^{-i+1} \implies c_1 = \lfloor B(x - \lfloor x \rfloor) \rfloor$$

Metodo della
Moltiplicazioni per la
Base

e le altre cifre possono essere estratte allo stesso modo. Quindi se per estrarre le cifre della parte intera usavamo un metodo basato sulle divisioni ripetute per la base, in questo caso usiamo un metodo basato su *moltiplicazioni* ripetute per la base. Vediamo un esempio, prima in decimale, e poi di conversione da decimale a binario.

Esempio 15: Usiamo $x = 3.78125$. La parte frazionaria è $x - \lfloor x \rfloor = 0.78125$, adesso $0.78125 \cdot 10 = 7.8125$ e $\lfloor 7.8125 \rfloor = 7$. Adesso ripetiamo con la parte frazionaria 0.8125 , *moltiplicando* per la base estraiamo le cifre da sinistra verso destra. Se invece di moltiplicare per 10, usiamo la base in cui vogliamo scrivere le cifre, ad esempio in binario otteniamo:

$$\begin{aligned} 0.78125 \cdot 2 &= 1.56250 \\ (1.56250 - 1) \cdot 2 &= 1.125 \\ 0.125 \cdot 2 &= 0.25 \\ 0.25 \cdot 2 &= 0.5 \\ 0.5 \cdot 2 &= 1.0 \end{aligned}$$

quindi prendendo le parti intere si ha: $(.78125)_{10} = (.11001)_2$, verifichiamolo: $1/2 + 1/4 + 1/32 = 0.5 + 0.25 + 0.03125 = 0.78125$. Notiamo che se fissiamo a 5 il numero di bit della parte frazionaria, il numero immediatamente più grande di $.11001$ è $.11010 = 1/2 + 1/4 + 1/16 = 0.8125$, quindi ogni numero reale x con $0.78125 < x < 0.8125$ non può essere rappresentato esattamente, ma dovrà essere approssimato.

Notiamo che il procedimento sopra può generare una sequenza infinita di bit, che dovrà essere necessariamente troncata, ad esempio proviamo a convertire in binario $17/5 = 3.4$ usando 3 bit per la parte intera e 5 bit per la parte frazionaria: la parte intera è $3_{10} = 011_2$, la parte frazionaria 0.4 in binario sarà:

$$0.4 \cdot 2 = 0.8$$

⁹Il lettore può trovare interessante, a questo punto, rivedere la definizione di numero reale studiata nel corso di Analisi I

$$0.8 \cdot 2 = 1.6$$

$$0.6 \cdot 2 = 1.2$$

$$0.2 \cdot 2 = \mathbf{0.4}$$

notiamo che dopo 4 moltiplicazioni otteniamo di nuovo il valore iniziale 0.4; da questo momento in poi è evidente che si ripeterà la sequenza di cifre 0110 continuamente. Pertanto il valore esatto per la parte frazionaria sarebbe $\overline{0110}$. La rappresentazione troncata è $011.01101 = 3.40625$, con un *errore assoluto* pari a $3.4 - 3.40625 = 0.00625$ e relativo di $0.00625/3.4 \approx 1/1000$ (che segnala l'errore nella terza cifra della parte frazionaria).

1.2.1 Aritmetica in Virgola Fissa

La rappresentazione vista nella sezione precedente, costruita a partire dal Teorema 2 fissando a priori il numero di cifre per la parte intera, e per la parte frazionaria è detta *rappresentazione in virgola fissa*.

Il motivo per cui si usa tale dizione nasce da una semplice osservazione: una volta fissato il numero complessivo di cifre del sistema, ad esempio t cifre, ed il valore di k (il numero di cifre della parte intera), la parte frazionaria avrà $f = t - k$ cifre, ed il punto rimarrà sempre fisso tra le prime k e le rimanenti f cifre.

Notiamo che anche se dal punto di vista teorico, i sistemi¹⁰ in virgola fissa servano a rappresentare numeri frazionari, essi in pratica *manipolano interi*. Non è necessario memorizzare il valore di k o di f : dato $n \in \mathbb{N}$ con $n \leq B^t - 1$ esso rappresenterà il razionale $x = n/B^f$: ogni numero quindi viene memorizzato come un intero e considerato implicitamente diviso per B^f , f è detto anche *precisione* della rappresentazione. Ad esempio, se $t = 8$, e $k = 3$, il valore di $x = 345.67814$ verrà rappresentato con l'intero 34567814. In binario, questo significa che possiamo usare i numeri interi, rappresentati magari in complemento a due, per rappresentare numeri in virgola fissa. Di solito la dimensione t è un multiplo di 8, la dimensione esatta dipende da come è stata costruita l'ALU, quindi è essenzialmente legata al tipo di processore. Nei processori moderni, l'ALU può manipolare numeri interi di 32 o 64 bit (4 o 8 byte) pertanto, il valore di t è uno tra essi, il programmatore sceglie di solito il valore di f a seconda della precisione necessaria nei calcoli, e da questo deriva poi il range dei valori rappresentabili.

Questo sistema di rappresentazione è molto comodo se il processore non possiede unità dedicate per lavorare con numeri frazionari, ed era molto usata negli anni '70 ed '80 anche se presenta vari problemi che illustreremo immediatamente. Nei calcolatori moderni è stata rimpiazzata dallo standard IEEE 754 che usa una rappresentazione in *virgola mobile* (*floating-point*), tale rappresentazione è più efficiente nell'uso delle cifre, migliorando quindi la precisione dei calcoli, inoltre essa si presta facilmente ad essere implementata tramite un circuito dedicato, detto FPU (*floating-point unit* l'analogo dell'ALU per i calcoli non interi). La velocità di questi due circuiti, misurata in MIPS: **M**illion of **I**nteger operation **P**er **S**econd, e FLOPS: **F**loat **O**peration **P**er **S**econd, sono stati per molto tempo indicatori molto diffusi delle *performance* di un calcolatore, nell'ambito scientifico, dove si usano più frequentemente dati in virgola mobile, si guarda ancora oggi al numero di Giga¹¹ *Flops* o Tera *Flops* di un calcolatore. Vediamo adesso il problema principale legato alla rappresentazione in virgola fissa. Calcoliamo questi valori in decimale, il risultato infatti è simile cambiando la base.

MIPS
FLOPS

Fissiamo i valori di t, k, f , ad esempio $t = 8$, $k = 5$ e $f = 3$, il range dei valori rappresentabili nel caso di valori senza segno è il seguente: Il minimo è zero ed il valore massimo è dato da 99999.999 , quindi avremo $10^5 - 1$ per la parte intera e $0.999 = 1 - 0.001 =$

¹⁰Uso il plurale perché fissato t abbiamo vari sistemi al variare di k .

¹¹Giga = 10^9 , Tera = 10^{12} , sono prefissi standard (sistema internazionale) per indicare i multipli di un'unità di misura, come lo studente dovrebbe sapere dal corso di Fisica I

$1 - 10^{-3} = 1 - 10^{-f}$, quindi il massimo valore è $10^k - 10^{-f} = 100000 - 0.001 = 99999.999$. Supponiamo di voler rappresentare un numero intero come 100001, nonostante questo valore abbia solo 6 cifre, esso non è rappresentabile nel sistema sopra poichè la suddivisione tra parte intera e parte frazionaria è *rigida*, lo stesso vale per 0.323423, nonostante abbia solo 6 cifre esso non può essere rappresentato, infatti useremo una cifra nella parte intera per lo zero, e le 4 cifre restanti saranno sprecate, costringendoci a troncare la parte frazionaria a 3 cifre. La conclusione ovvia è che nei sistemi di rappresentazione in virgola fissa, lo spazio (le cifre) non sono usate in modo efficiente.

1.2.2 Numeri Floating-Point - IEEE 754

I problemi della rappresentazione in virgola fissa hanno portato allo sviluppo della notazione in virgola mobile che è probabilmente già conosciuta dallo studente come *rappresentazione in notazione scientifica*. Partendo dai problemi della notazione in virgola fissa, l'idea che vogliamo realizzare è la seguente: fissiamo il numero di cifre complessivo (t), vogliamo poter mettere la virgola dove ci pare (*floating*) in modo da poter usare le t cifre per la parte intera se serve, o per la parte frazionaria, a seconda del valore da rappresentare.

Per far questo, notiamo che se ho $x = 45.346$ posso spostare la virgola *lasciando invariato* il valore di x se e solo se tengo conto di tale spostamento, moltiplicando o dividendo x per una potenza opportuna della base:

$$45.346 = 4.5346 \cdot 10^1 = 0.45346 \cdot 10^2 = 453.46 \cdot 10^{-1} = 45346 \cdot 10^{-3}$$

ed in generale abbiamo un'infinità di modi per scrivere x . Recuperiamo l'unicità se decidiamo tra tutte le rappresentazioni di usare quella con l'esponente che porta un'unica cifra *diversa da zero* a sinistra del punto e le restanti tutte a destra, tale scelta viene detta *normalizzazione* del valore di x . Nel caso sopra useremo $x = 4.5346 \cdot 10^1$. Vale pertanto il seguente Teorema di rappresentazione:

Teorema 3 (Notazione in virgola mobile).

Dato un numero reale $x \in \mathbb{R} \neq 0$, ed una base $B > 1$ esistono:

- un segno $s = \text{sign}(x)$,
- un unico valore $m = c_0.c_{-1}c_{-2}\dots$ chiamato *mantissa*, formata da una sequenza (eventualmente infinita) di cifre con $0 \leq c_i < B$, e $c_0 \neq 0$,
- un esponente $e \in \mathbb{Z}$.

per cui vale:

$$x = (-1)^s m B^e = (-1)^s \cdot c_0 + \left(\sum_{i=-1}^{\infty} c_i B^{-i} \right) \cdot B^e$$

Il Teorema sopra può essere usato per definire un insieme *finito* di valori, detti *numeri di macchina* ed indicati con $\mathbb{F}(B, n, t, q)$ dove n è il numero complessivo di cifre usate, t è il numero di cifre usate per la mantissa e q il numero di cifre per l'esponente. Notiamo che lo zero non fa parte di questo insieme.

Sistema IEEE-754

Lo standard IEEE-754 usato nei computer moderni estende $\mathbb{F}(B, n, t, q)$ con lo zero (rappresentato in modo particolare), ed altri valori, tra cui: un insieme di numeri *de-normalizzati* (con $c_0 = 0$) e alcuni speciali come $\pm\infty$ ed un *non-valore* detto *Nan* (Not a Number) la cui utilità verrà spiegata successivamente.

Vediamo intanto la rappresentazione di un numero in IEEE-754: prima di tutto fissiamo la base, vi sono processori che lavorano in base 10, ma la base prevalente è quella binaria; il

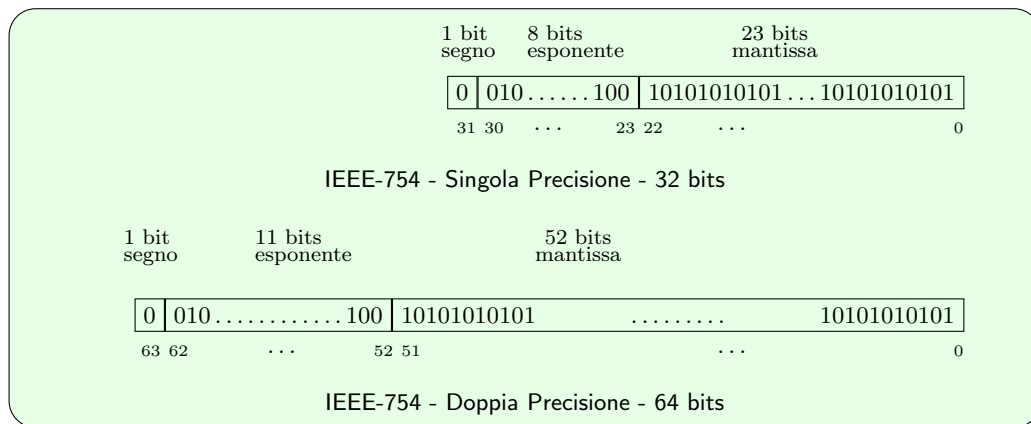
numero complessivo di cifre può variare tra tre dimensioni: $n = 32$ bit o *singola precisione*, 64 bit o *doppia precisione* e in alcuni casi *quadrupla precisione* 128 bit.

Il segno (s) è sempre rappresentato tramite il bit più significativo (quindi la notazione è simile alla modulo e segno), subito dopo viene rappresentato l'esponente. Quest'ultimo è un valore con segno, e lo si scrive con una rappresentazione con scostamento (vedi Sezione 1.1.3). Infine vi sono i bit che rappresentano la mantissa, che si rappresentano con una particolarità: nel caso di numeri normalizzati, poichè $c_0 \neq 0$, allora si ha $c_0 = 1$ (perche siamo in base 2). La cifra c_0 essendo costante, non è memorizzata esplicitamente, quindi la mantissa contiene le rimanenti cifre $c_{-1} \dots c_{-t}$.

Dato il segno s , la mantissa m e l'esponente e , il valore rappresentato (per i numeri normalizzati) è:

$$x = (-1)^s (1 + m) 2^{e-\text{bias}} \quad (1.4)$$

Sotto vediamo il formato IEEE-754 nel caso di numeri in singola o doppia precisione.



Esempio 16: Convertire -5.828125 in IEEE-754 in singola precisione:

Cominciamo con la parte intera: $5 = 4 + 1 = (101)_2$, (potevamo usare anche il metodo della divisione ripetuta per 2). Per la parte frazionaria abbiamo, seguendo il metodo delle moltiplicazioni per la base: $0.828125 \times 2 = 1.65625$, quindi la prima cifra è 1, poi ripetiamo su 0.65625 e così via, fino ad ottenere 1101010. Quindi abbiamo $5.828125_{10} = 1.011101010 \times 2^2$ normalizzato.

L'esponente va codificato in scostamento a $2^7 - 1 = 127$, quindi: $2 + 127 = 129$ e in binario $129 = 128 + 1 = 10000001$ infine il segno è negativo quindi in definitiva abbiamo

$$-5.828125 = 1 \ 10000001 \ 011101010000000000000000$$

in esadecimale più compattamente $0xC0BA8000$ (il prefisso '0x' è un'altra notazione per indicare che il numero adiacente è in base 16).

Poichè l'esponente è di 8 bit, il range di valori va da 0 a 255. I valori estremi sono usati per rappresentare valori particolari, come vedremo tra poco, mentre i valori validi per l'esponente dei numeri normalizzati varia da 1 a 254, e corrisponde al range di esponenti validi $-126 \dots 127$. I valori (positivi e negativi) più vicini allo zero sono pertanto $1.0 \times 2^{-126} \approx \pm 1.17549 \times 10^{-38}$ mentre il valore più lontano da zero è fatto da una mantissa di tutti 1 ed esponente pari a 127 quindi:

$$2^{127} \sum_{i=0}^{23} 2^{-i} = 2^{104} \sum_{i=0}^{23} 2^i = 2^{104} (2^{24} - 1) \approx 3.4028 \times 10^{38}$$

Abbiamo visto che se l'esponente vale 0 o 255, i valori rappresentati sono particolari:

- Lo zero non è un numero normalizzato ($c_0 = 0$) e viene rappresentato con tutti i bit a zero. Notiamo che come nella rappresentazione con mantissa e segno, esiste un $+0 = 0x00000000$ ed un $-0 = 0x80000000$ a seconda del valore del bit del segno.
- Quando l'esponente è posto con tutti i bit ad uno, e se la mantissa è tutta a zero, questo non rappresenta come dovrebbe $\pm 1.0 \times 2^{126}$ ma rappresenta a seconda del valore del bit di segno due valori speciali $\pm\infty$.
- Se l'esponente è sempre con tutti i bit ad uno, i.e. $126(+127)$ ma la mantissa è diversa da zero, allora abbiamo un particolare valore detto NAN = *Not a Number* per indicare il risultato di operazioni non definite: $1/0$ o $\sqrt{-1}$.

Resta la sola possibilità ancora non considerata di un esponente a zero e di una mantissa diversa da zero. Questi numeri sono chiamati *denormalizzati* e in pratica corrispondono alla stessa espressione dell'Equazione 1.4 ma con il bit implicito $c_0 = 0$ e non $c_0 = 1$ (quindi non $(1+m)$ ma semplicemente m). Questi numeri, sono chiaramente più piccoli dei numeri normalizzati o meglio più vicini allo zero, e servono esattamente a ridurre l'errore in quell'intervallo.

Per lo studente di matematica la comprensione delle limitazioni legate alla rappresentazione con numeri floating-point è molto importante. Per approfondire ulteriormente al di là di queste note, un riferimento interessante è il seguente: "What Every Scientist Should Know About Floating-Point Arithmetic" (scaricabile cliccando sul nome dell'articolo) o all'indirizzo: <http://www.validlab.com/goldberg/paper.pdf>.

Abbiamo concluso la parte legata alla rappresentazione in binario di numeri interi, naturali e con segno, e di numeri non interi, frazionari, e reali (approssimati tramite numeri floating-point a precisione limitata). Nella prossima sezione vedremo come si rappresentano le informazioni di tipo non numerico.

1.3 Testi, Immagini, Musica

Come sappiamo, i calcolatori moderni possono manipolare oltre che numeri, anche informazioni più complesse: testi, in varie lingue, immagini fisse in bianco e nero e a colori, video e audio. In questa sezione vedremo come è possibile trasformare queste informazioni in forma binaria.

In tutti i casi, le rappresentazioni verranno costruite tramite *codifiche* che trasformano quelle informazioni di cui sopra in sequenze di numeri interi. Dal punto di vista matematico vedremo che queste codifiche degli *isomorfismi*, molto semplici da applicare.

Testi

Cominciamo con l'informazione non-numerica di tipo più semplice, *un testo*. I testi, formalmente sono *stringhe*, cioè sequenze finite di caratteri, dove i possibili caratteri sono presi da un insieme finito detto *alfabeto*. Anche questo testo che leggete soddisfa questa definizione, in quanto, possiamo vederlo come una lunghissima sequenza di caratteri (dell'alfabeto Italiano, unito a tutti i caratteri di interpunzione e allo spazio). Poiché l'insieme di simboli dell'alfabeto è per ipotesi *finito*, possiamo associare ad ogni simbolo un numero intero positivo, ad esempio potremmo definire: A= 65, B= 66 ... Z= 132, etc; come mai non ho associato A= 1, etc; dal punto di vista teorico qualunque biezione tra i simboli dell'alfabeto ed un opportuno sottoinsieme di $\mathbb{N}$ andrebbe benissimo, ma quella fatta sopra è un estratto di una codifica standard, per i testi, chiamata *Codifica ASCII*. Il lettore può consultare Internet per avere la tabella completa (ad esempio cercando la voce ASCII su Wikipedia), e troverà le corrispondenze tra ogni possibile simbolo della tastiera di un calcolatore, compresi quelli non stampabili come lo spazio e il *ritorno carrello*, e molti altri.

The ASCII code

American Standard Code for Information Interchange

ASCII control characters			ASCII printable characters						Extended ASCII characters											
DEC	HEX	Simbolo ASCII	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo	DEC	HEX	Simbolo
00	00h	NULL (carattere nullo)	32	20h	spazio	64	40h	@	96	60h	`	128	80h	À	160	A0h	à	192	C0h	À
01	01h	SOH (inizio encabezado)	33	21h	!	65	41h	A	97	61h	a	129	81h	Á	161	A1h	á	193	C1h	Á
02	02h	STX (inizio testo)	34	22h	"	66	42h	B	98	62h	b	130	82h	Â	162	A2h	â	194	C2h	Â
03	03h	ETX (fin de texto)	35	23h	#	67	43h	C	99	63h	c	131	83h	Ã	163	A3h	ã	195	C3h	Ã
04	04h	EOT (fin transmisión)	36	24h	\$	68	44h	D	100	64h	d	132	84h	Ä	164	A4h	ä	196	C4h	Ä
05	05h	ENQ (enquiry)	37	25h	%	69	45h	E	101	65h	e	133	85h	Å	165	A5h	å	197	C5h	Å
06	06h	ACK (acknowledgement)	38	26h	&	70	46h	F	102	66h	f	134	86h	Ä	166	A6h	ä	198	C6h	Ä
07	07h	BEL (bello)	39	27h	'	71	47h	G	103	67h	g	135	87h	Å	167	A7h	å	199	C7h	Å
08	08h	BS (retroceso)	40	28h	(	72	48h	H	104	68h	h	136	88h	Ä	168	A8h	ä	200	C8h	Ä
09	09h	HT (tab horizontal)	41	29h	)	73	49h	I	105	69h	i	137	89h	Ä	169	A9h	ä	201	C9h	Ä
10	0Ah	LF (salto de linea)	42	2Ah	*	74	4Ah	J	106	6Ah	j	138	8Ah	Ä	170	AAh	ä	202	CAh	Ä
11	0Bh	VT (tab vertical)	43	2Bh	+	75	4Bh	K	107	6Bh	k	139	8Bh	Ä	171	ABh	ä	203	CBh	Ä
12	0Ch	FF (form feed)	44	2Ch	,	76	4Ch	L	108	6Ch	l	140	8Ch	Ä	172	ACH	ä	204	Ch	Ä
13	0Dh	CR (retorno de carro)	45	2Dh	-	77	4Dh	M	109	6Dh	m	141	8Dh	Ä	173	ADh	ä	205	CDh	Ä
14	0Eh	SO (shift Out)	46	2Eh	.	78	4Eh	N	110	6Eh	n	142	8Eh	Ä	174	Aeh	ä	206	CEh	Ä
15	0Fh	SI (shift in)	47	2Fh	/	79	4Fh	O	111	6Fh	o	143	8Fh	Ä	175	Afh	ä	207	CFh	Ä
16	10h	DLE (data link escape)	48	30h	0	80	50h	P	112	70h	p	144	90h	Ä	176	B0h	ä	208	D0h	Ä
17	11h	DC1 (device control 1)	49	31h	1	81	51h	Q	113	71h	q	145	91h	Ä	177	B1h	ä	209	D1h	Ä
18	12h	DC2 (device control 2)	50	32h	2	82	52h	R	114	72h	r	146	92h	Ä	178	B2h	ä	210	D2h	Ä
19	13h	DC3 (device control 3)	51	33h	3	83	53h	S	115	73h	s	147	93h	Ä	179	B3h	ä	211	D3h	Ä
20	14h	DC4 (device control 4)	52	34h	4	84	54h	T	116	74h	t	148	94h	Ä	180	B4h	ä	212	D4h	Ä
21	15h	NAK (negative acknowle.)	53	35h	5	85	55h	U	117	75h	u	149	95h	Ä	181	B5h	ä	213	D5h	Ä
22	16h	SYN (synchronous idle)	54	36h	6	86	56h	V	118	76h	v	150	96h	Ä	182	B6h	ä	214	D6h	Ä
23	17h	ETB (end of trans. block)	55	37h	7	87	57h	W	119	77h	w	151	97h	Ä	183	B7h	ä	215	D7h	Ä
24	18h	CAN (cancel)	56	38h	8	88	58h	X	120	78h	x	152	98h	Ä	184	B8h	ä	216	D8h	Ä
25	19h	EM (end of medium)	57	39h	9	89	59h	Y	121	79h	y	153	99h	Ä	185	B9h	ä	217	D9h	Ä
26	1Ah	SUB (substitute)	58	3Ah	:	90	5Ah	Z	122	7Ah	z	154	9Ah	Ä	186	BAh	ä	218	DAh	Ä
27	1Bh	ESC (escape)	59	3Bh	;	91	5Bh	[	123	7Bh	{	155	9Bh	Ä	187	BBh	ä	219	DBh	Ä
28	1Ch	FS (file separator)	60	3Ch	<	92	5Ch	\	124	7Ch		156	9Ch	Ä	188	BCh	ä	220	DCh	Ä
29	1Dh	GS (group separator)	61	3Dh	=	93	5Dh	]	125	7Dh	}	157	9Dh	Ä	189	BDh	ä	221	DDh	Ä
30	1Eh	RS (record separator)	62	3Eh	>	94	5Eh	^	126	7Eh	~	158	9Eh	Ä	190	BEh	ä	222	DEh	Ä
31	1Fh	US (unit separator)	63	3Fh	?	95	5Fh	_				159	9Fh	f	191	BFh	ä	223	DFh	Ä
127	7Fh	DEL (delete)																		

Figura 1.2: Tabella ASCII per la rappresentazione numerica dei caratteri (stampabili e non).

Quindi un testo è una sequenza di caratteri codificati come interi, tramite il codice ASCII, quest'ultimo associa ad ogni carattere un byte (otto bit). Notiamo che, poichè un byte sono otto bit, il numero massimo di caratteri diversi che possiamo rappresentare in ASCII sono esattamente 256. L'ASCII è stato per tanto tempo la sola codifica standard, ma nell'ultimo decennio esso è stato superato da codifiche più complesse come UTF-8/Unicode. Non entreremo nel dettaglio di questi aspetti, noi considereremo nel corso di Programmazione i testi come semplici sequenze di interi (ottenuti tramite la tabella ASCII).

Musica

Un suono formalmente è *un'onda* rappresentabile come una funzione continua del tempo, descritta da opportuni parametri come l'ampiezza, la frequenza, etc. E' abbastanza chiaro al lettore, vista la sua formazione matematica, e quanto detto finora, che una funzione *continua* non potrà mai essere rappresentata esattamente all'interno di un calcolatore. Innanzitutto il suono deve avere lunghezza finita, ad esempio t secondi. Possiamo trasformare il segnale audio in modo che esso sia descrivibile da una funzione continua $Sound : T \mapsto D$, dove $T = [0, t]$ è il tempo, e $D = [-A, A]$ con $A \in \mathbb{R}$ è l'ampiezza massima del segnale.

A questo punto dobbiamo attraversare due fasi di *discretizzazione* per trasformare il dominio e il codominio da insiemi *continui* a *discreti* (ed in particolare *finiti*). La discretizzazione del dominio (il tempo) si chiama nello specifico *campionamento del segnale*, e consiste nella forma più semplice nel decidere i punti in cui valutiamo la funzione *Sound*, a passi regolari, ad esempio M valutazioni nell'intervallo $[0, 1]$ (M campioni/secondo). Esiste un Teorema (di Nyquist) che dice che se la frequenza di campionamento è il doppio della massima frequenza presente in *Sound*, allora se campiniamo con frequenza maggiore o uguale a questo valore, il segnale discreto così ottenuto può essere *interpolato* per ricostruire senza perdita il segnale originale.

Ad esempio, l'udito umano è sensibile alle frequenze fino a $22kHz$, pertanto se campiniamo con una frequenza di $44kHz$ un segnale audio, (quindi se raccogliamo circa 44000 campioni al secondo) otteniamo un segnale *discreto* che però contiene in un certo senso tutta l'informazione del segnale continuo originario. Ecco perché se notate l'etichetta di un CD-Audio, troverete che il segnale è campinato esattamente a $44.1kHz$.

Nella Figura accanto la fase di campionamento corrisponde a valutare la funzione nei punti indicati. Adesso dobbiamo discretizzare il codominio, e questa fase si chiama *quantizzazione*, infatti il valore della funzione è una quantità continua nel codominio $[-A, A]$ e come tale non rappresentabile senza errori. La quantizzazione, decide quanti bit usare per rappresentare un valore del codominio, ad esempio se usiamo 16 bit, abbiamo $2^{16} = 65536$ il valore 0 corrisponderà a $-A$, il valore 65535 a A , con i valori su una griglia di passo $1/65536$. A questo punto, possiamo memorizzare i 44100 campioni per secondo (frequenza di campionamento di 44.1 kHz, dove per ogni campione useremo due byte (16 bit)). In questo modo, se prendiamo una canzone di lunghezza media (circa 3 minuti e mezzo) abbiamo $3.5 \times 60 = 210s$ per ogni secondo usiamo 2×44100 byte, per un totale di 18522000, cioè $18522000 / (1024 \times 1024) = 17.66$ MB (MegaByte). Normalmente noi memorizziamo queste informazioni in forma compressa (ad esempio tramite i formati con perdita .mp3 o senza perdita .flac), impiegando una frazione minore di spazio.

Come si vede, anche in questo caso, abbiamo trasformato un'informazione di tipo non numerico in una sequenza di interi.

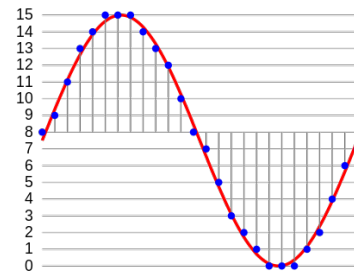


Figura 1.3: Campionamento e Quantizzazione di un Segnale Continuo ad una dimensione (Audio)

Immagini, Video

Il procedimento impiegato per codificare la musica, può essere usato allo stesso modo con segnali bidimensionali come le immagini. In questo caso, la funzione a due variabili $Img(x, y) : [0, MAXX] \times [0, MAXY] \mapsto Colors$ associa ad ogni punto del piano, un valore che rappresenta il colore del segnale visivo in quel punto. Il procedimento di campionamento, consiste nel determinare una *griglia* di dimensione finita, e valutare la funzione nei punti della griglia. Il numero complessivo di punti verrà chiamato *risoluzione* dell'immagine, ed ogni punto viene chiamato *pixel*, ad esempio, nei televisori attuali con risoluzione *4kUHD* si hanno 3840×2160 pixels. La fase di quantizzazione, consiste nel decidere come il colore di un punto viene rappresentato: se usiamo una *sintesi additiva* il colore sarà la somma di alcune frequenze primarie, in particolare: la somma di una certa quantità di rosso, verde e blu, determina un sistema chiamato RGB (Red, Green, Blue). Se per specificare queste quantità si usa un byte per ognuna di esse (24 bit in totale), otteniamo un totale di $256 \times 256 \times 256 \approx 16M$ sfumature possibili. Le immagini HDR dei televisori moderni hanno uno spazio di colore ancora più ampio.

Una immagine catturata con uno smartphone con una fotocamera digitale di 10Mpixel, occuperà 4 byte (si preferisce dedicare un numero di byte pari ad ogni punto, quindi ad i 3 byte per il rosso, verde, e blu, se ne aggiunge un altro non usato) per pixel, per un totale non compresso di $10 \times 1024 \times 1024 \times 4 \approx 41MB$. Se confrontate questo numero con la dimensione media di un'immagine compressa in formato .jpg (in media circa 1MB) capite quanto siano efficaci gli algoritmi di compressione usati.

Quindi, se una canzone dentro il vostro calcolatore è rappresentata da una sequenza di interi, una immagine sarà una *matrice* di (quadruple) di interi.